

Busca em profundidade p/ digrafos

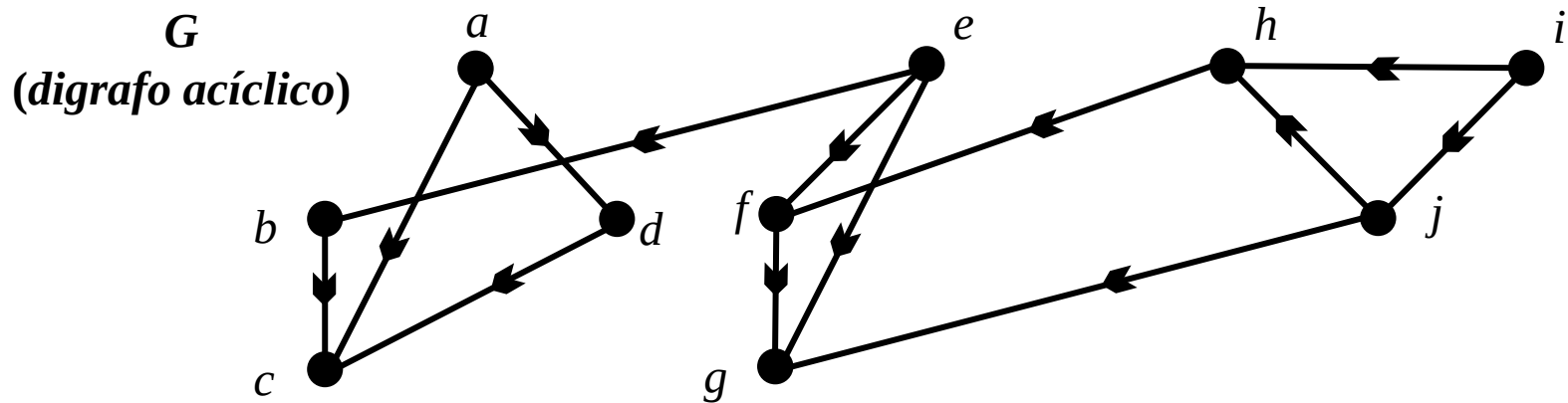
Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .

- Um digrafo G é **acíclico** se G não contém ciclos direcionados.
- Uma **ordenação topológica** de um digrafo acíclico G é uma ordenação $v_1 v_2 v_3 \dots v_{n-1} v_n$ dos vértices de G com a seguinte propriedade:

“se existe uma aresta em G de v_i a v_j então $i < j$ ”

Busca em profundidade p/ digrafos

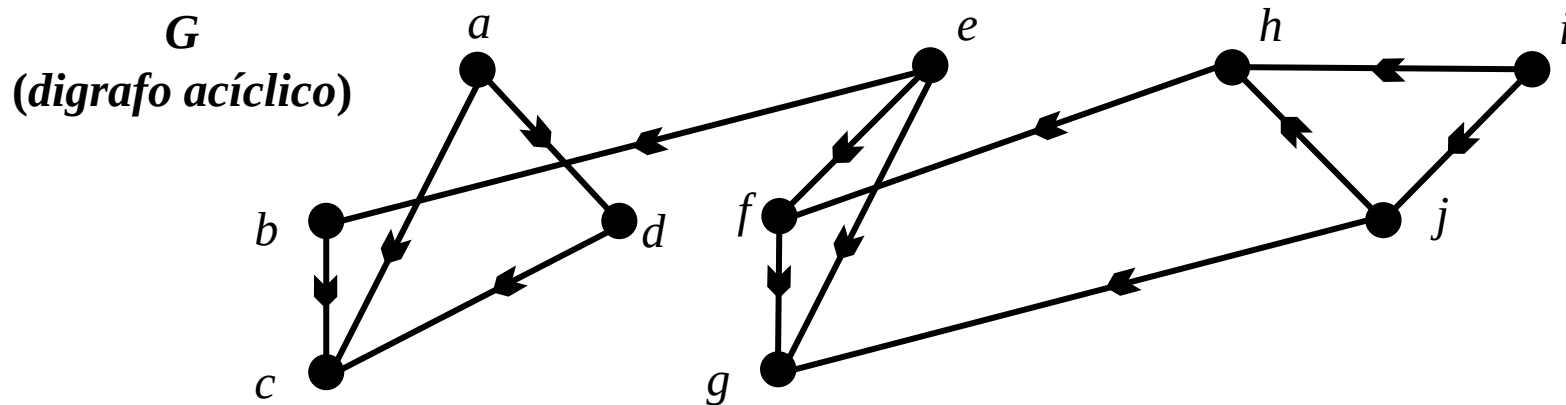
Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .



Uma ordenação topológica de G : ***i j h e f g a b d c***

Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



Uma ordenação topológica de G : ***i j h e f g a b d c***

Outra ordenação topológica de G : ***e i j h a f g d b c***

Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .

Como decidir se um dado digrafo é acíclico?

Basta rodar uma busca em profundidade sobre G . Se esta busca não produzir nenhuma **aresta de retorno**, então G é acíclico.

Se uma busca sobre G não produz aresta de retorno, então nenhuma outra busca produzirá aresta de retorno!

Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .

Aplicações da ordenação topológica

- **Escalonamento de atividades:** cada vértice representa uma atividade, e cada aresta direcionada de a para b indica que a atividade b só pode ser executada **depois** da atividade a . A ordenação topológica fornece portanto uma sequência viável para a realização das atividades.

Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .

Aplicações da ordenação topológica

- **Escalonamento de tarefas em um único processador:** cada vértice representa uma **tarefa**, e cada aresta direcionada de **a** para **b** indica que a tarefa **a** deve enviar um dado para a tarefa **b** (a tarefa **b** só pode iniciar sua execução depois que a tarefa **a** enviar o dado). Dado que existe um **único processador** para executar as tarefas, a ordenação topológica fornece uma sequência de execução das tarefas no processador.

Busca em profundidade p/ digrafos

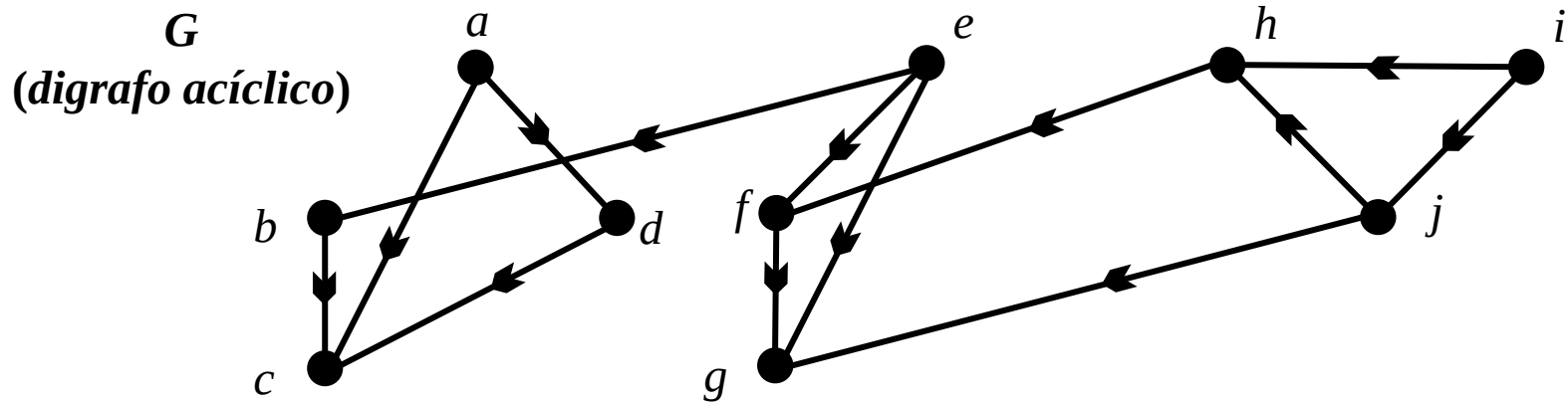
Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .

Algoritmo para ordenação topológica de um digrafo G :

1. Rodar uma busca em profundidade sobre G . Se esta busca produziu alguma aresta de retorno, então pare: G não é acíclico. Caso contrário, vá para o passo 2.
2. Ordene os valores das **profundidades de saída** geradas pela busca em **ordem decrescente**.
3. Faça v_1 como o vértice de maior PS , v_2 como o vértice de segunda maior PS , e assim por diante.
4. A ordenação $v_1 v_2 v_3 \dots v_{n-1} v_n$ obtida no passo 3 é uma ordenação topológica!

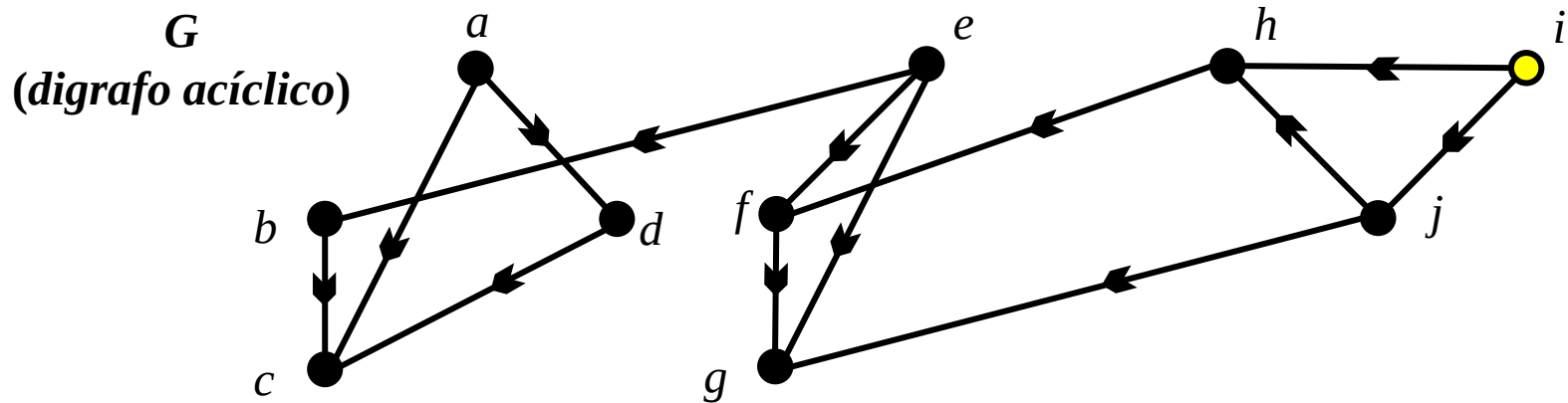
Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .

[illegible]

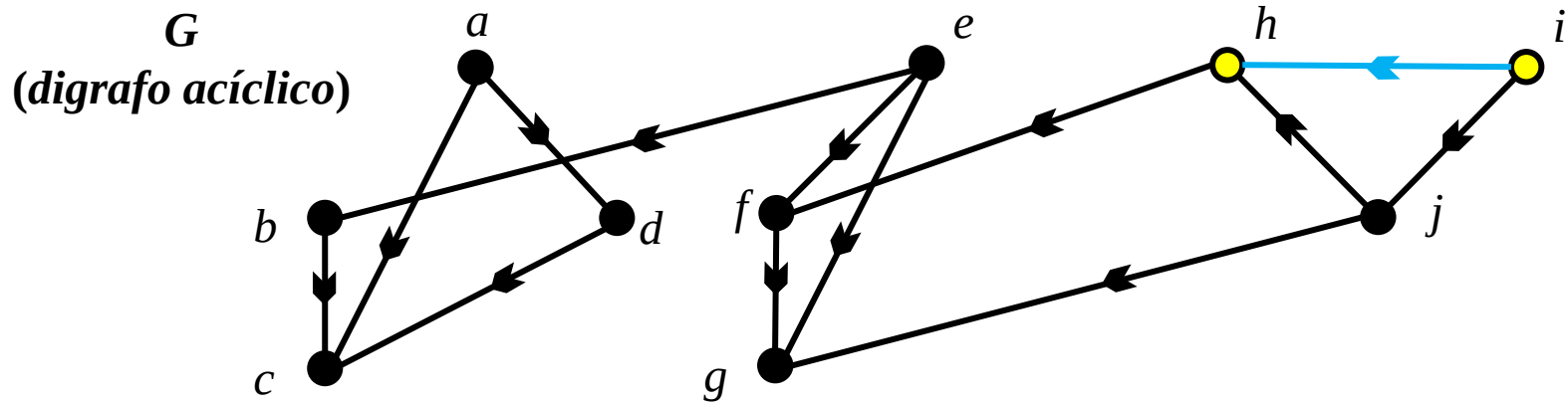
Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .

[illegible]

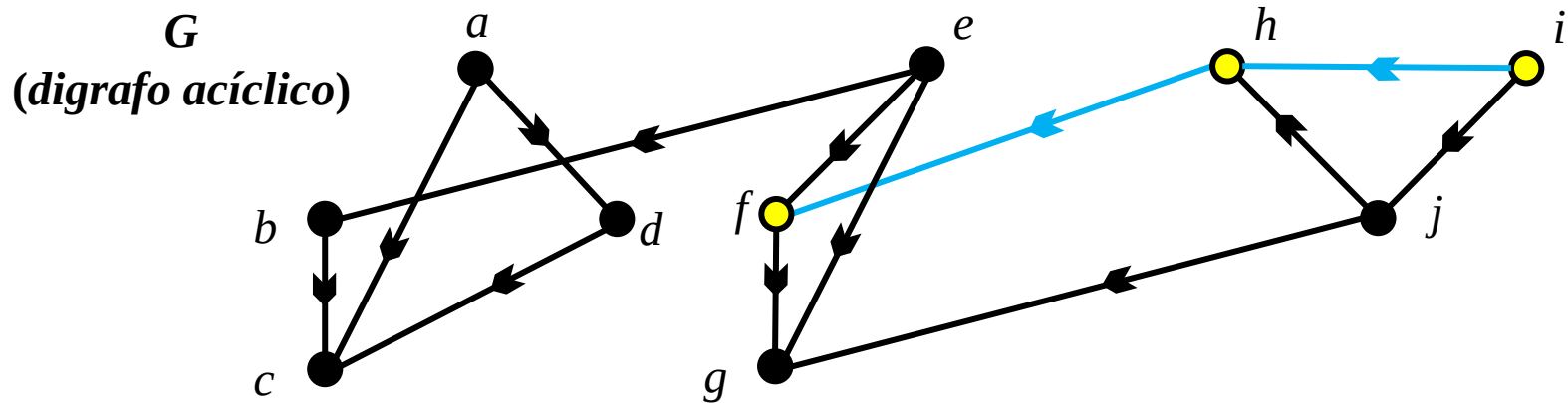
Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .

[illegible]

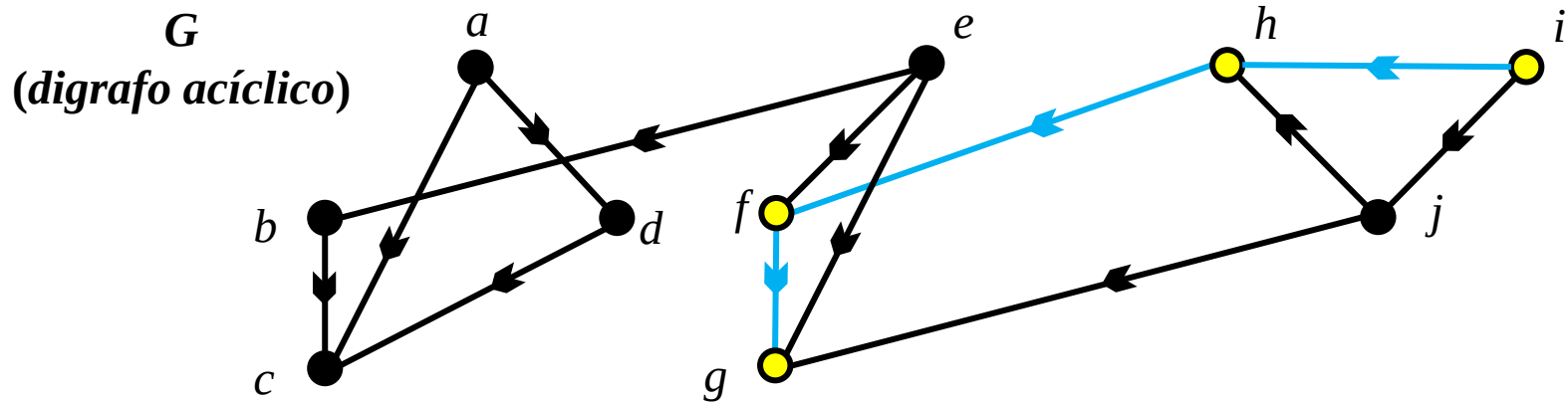
Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .

[illegible]

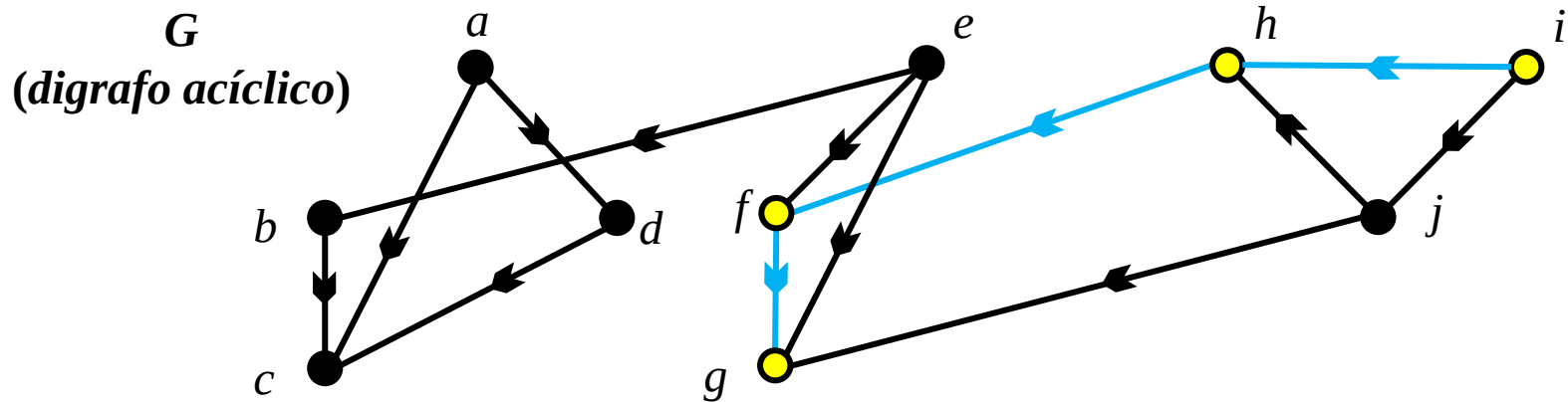
Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .

[illegible]

Busca em profundidade p/ digrafos

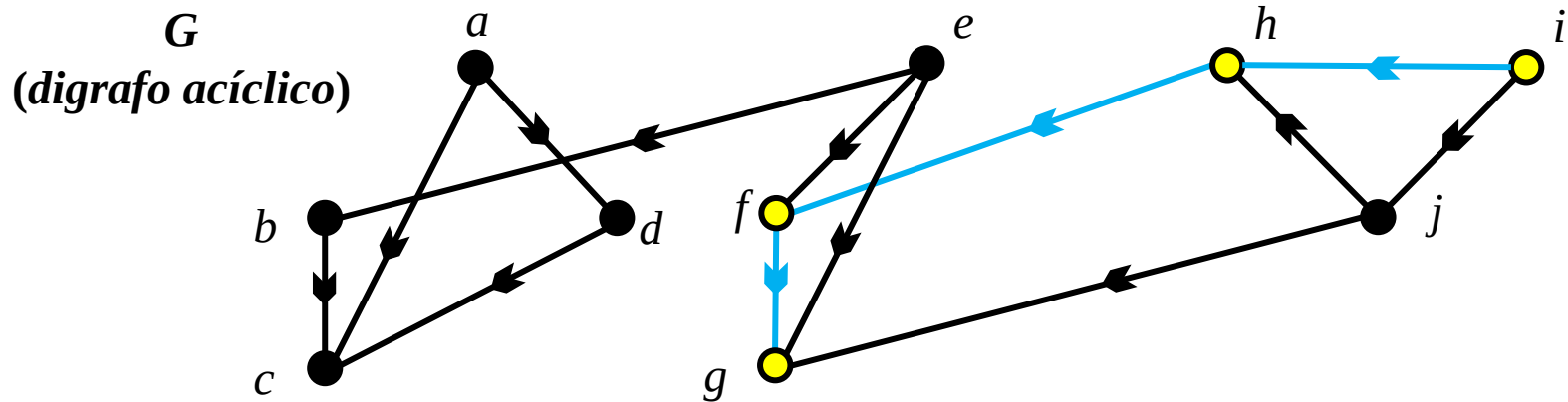
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	0	0	0	0	3	4	2	1	0
$PS(v)$	0	0	0	0	0	0	5	0	0	0

Busca em profundidade p/ digrafos

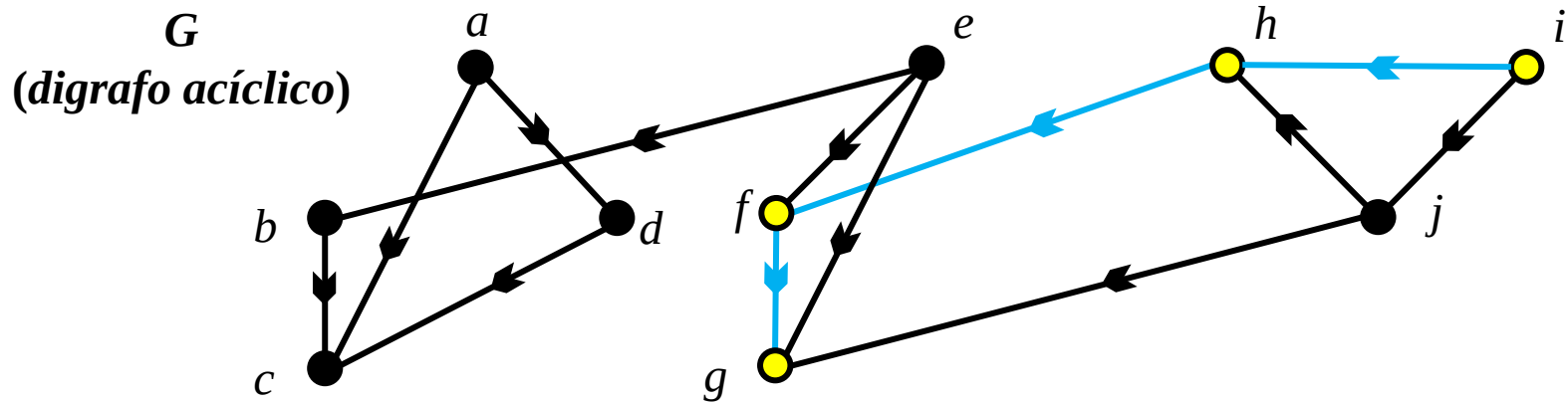
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	0	0	0	0	3	4	2	1	0
$PS(v)$	0	0	0	0	0	6	5	0	0	0

Busca em profundidade p/ digrafos

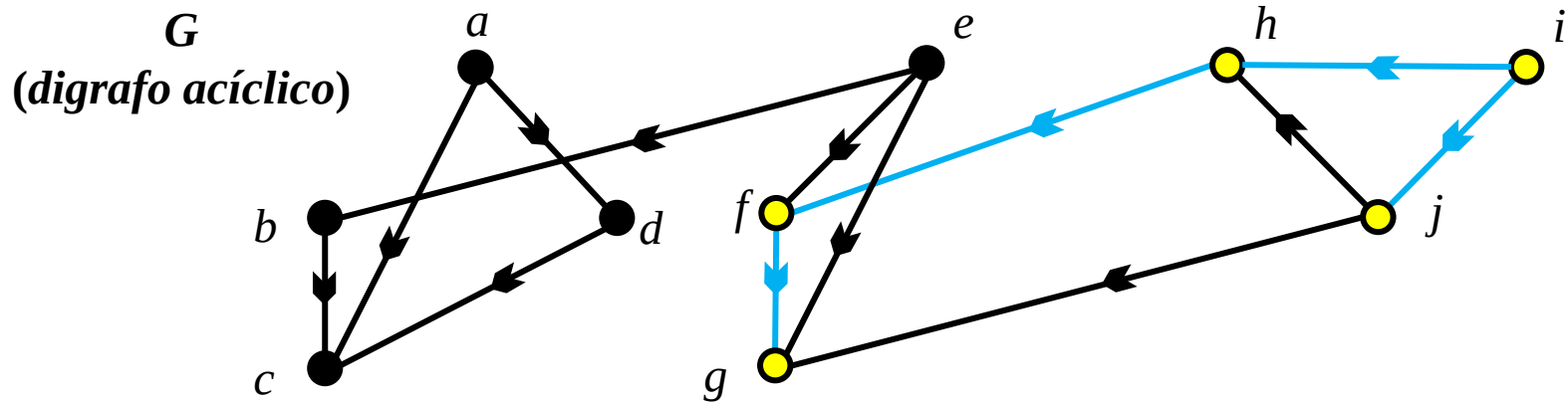
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	0	0	0	0	3	4	2	1	0
$PS(v)$	0	0	0	0	0	6	5	7	0	0

Busca em profundidade p/ digrafos

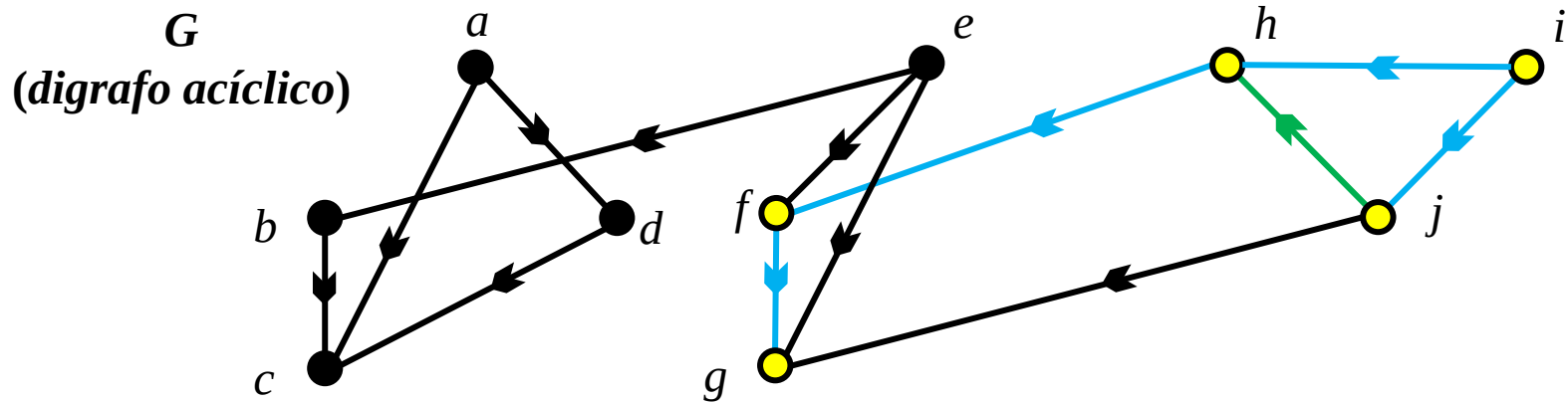
Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	0	0	0	0	3	4	2	1	8
$PS(v)$	0	0	0	0	0	6	5	7	0	0

Busca em profundidade p/ digrafos

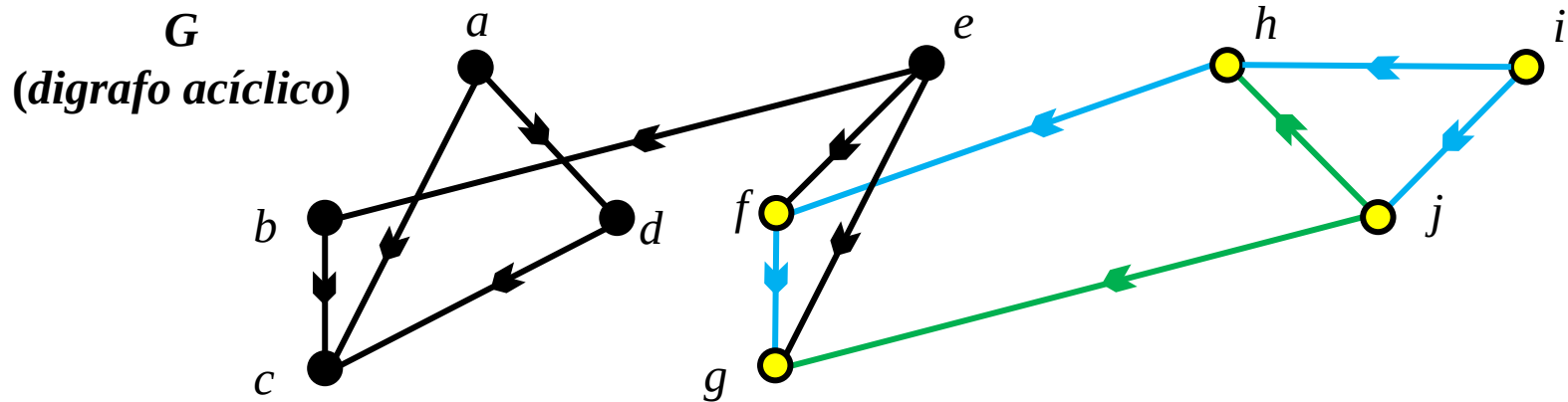
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	0	0	0	0	3	4	2	1	8
$PS(v)$	0	0	0	0	0	6	5	7	0	0

Busca em profundidade p/ digrafos

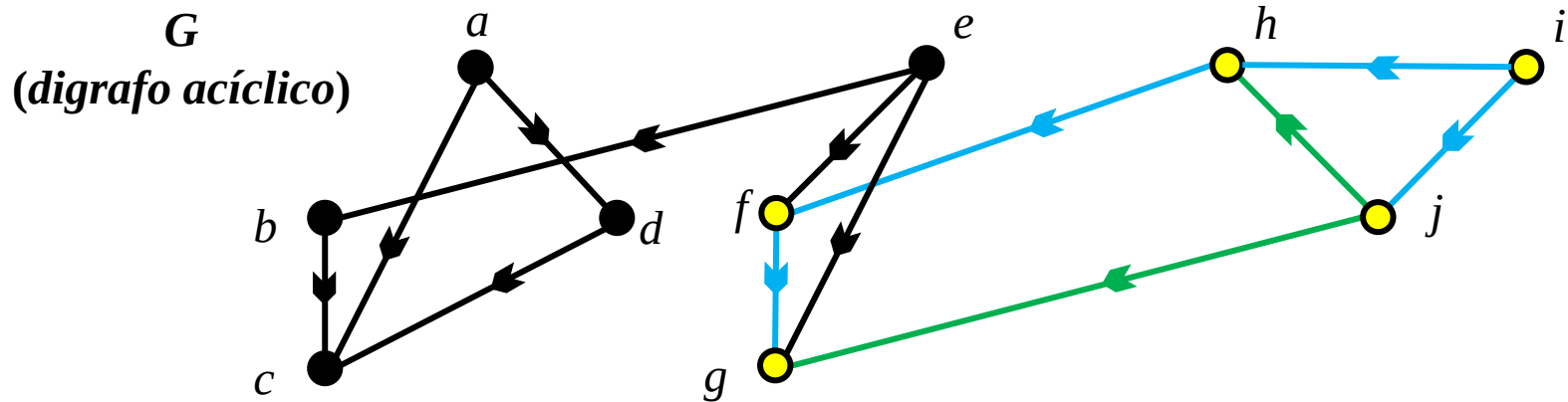
Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	0	0	0	0	3	4	2	1	8
$PS(v)$	0	0	0	0	0	6	5	7	0	0

Busca em profundidade p/ digrafos

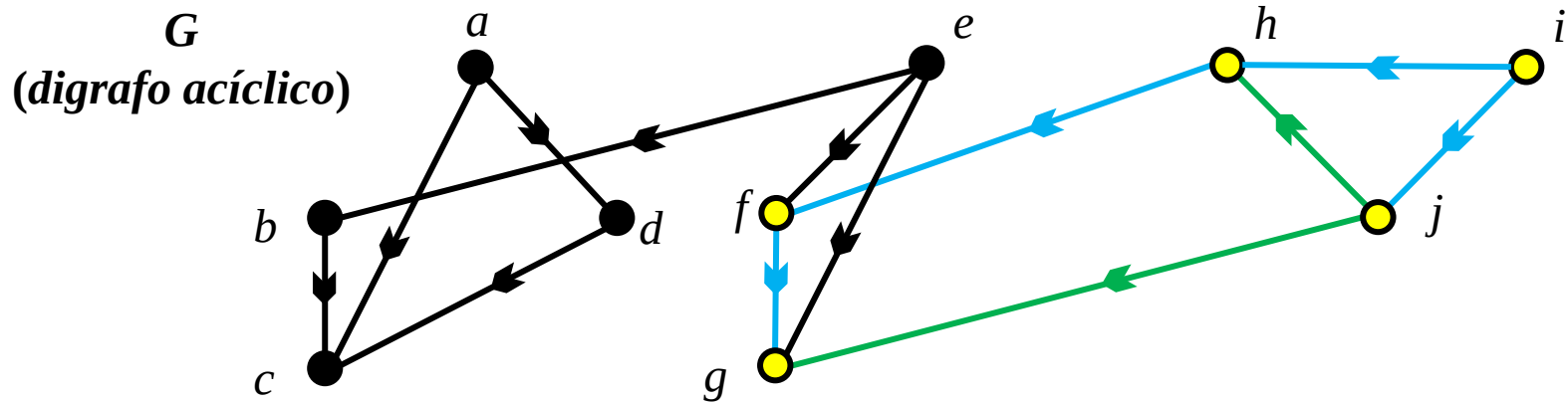
Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	0	0	0	0	3	4	2	1	8
$PS(v)$	0	0	0	0	0	6	5	7	0	9

Busca em profundidade p/ digrafos

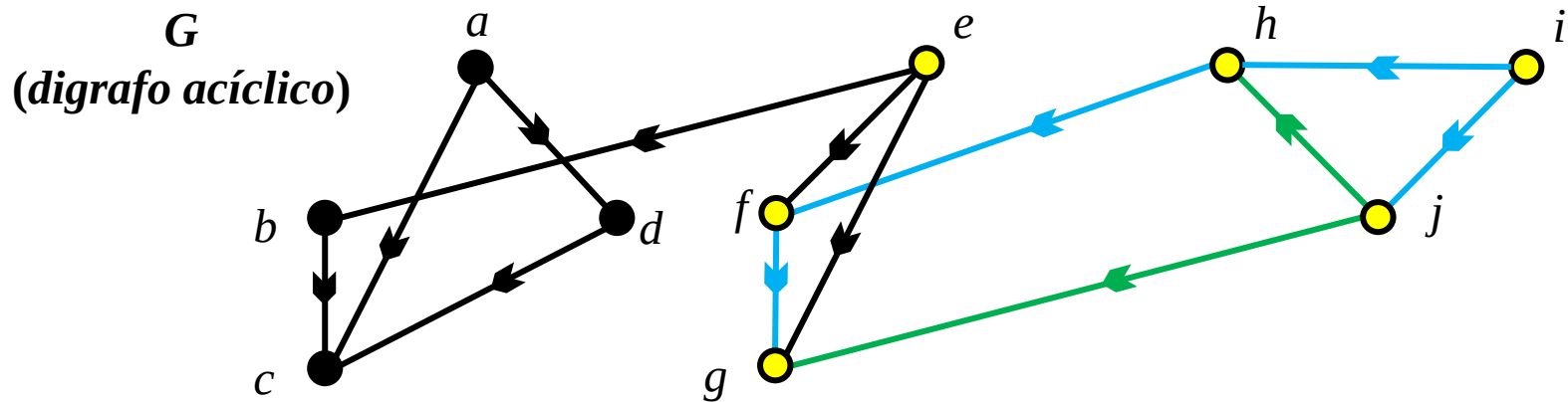
Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	0	0	0	0	3	4	2	1	8
$PS(v)$	0	0	0	0	0	6	5	7	10	9

Busca em profundidade p/ digrafos

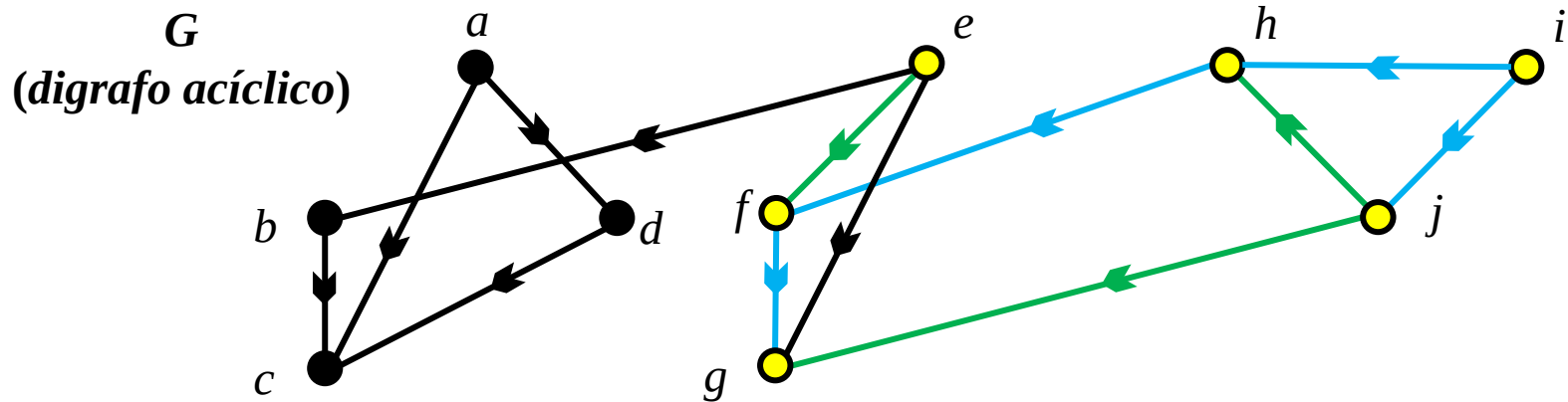
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	0	0	0	11	3	4	2	1	8
$PS(v)$	0	0	0	0	0	6	5	7	10	9

Busca em profundidade p/ digrafos

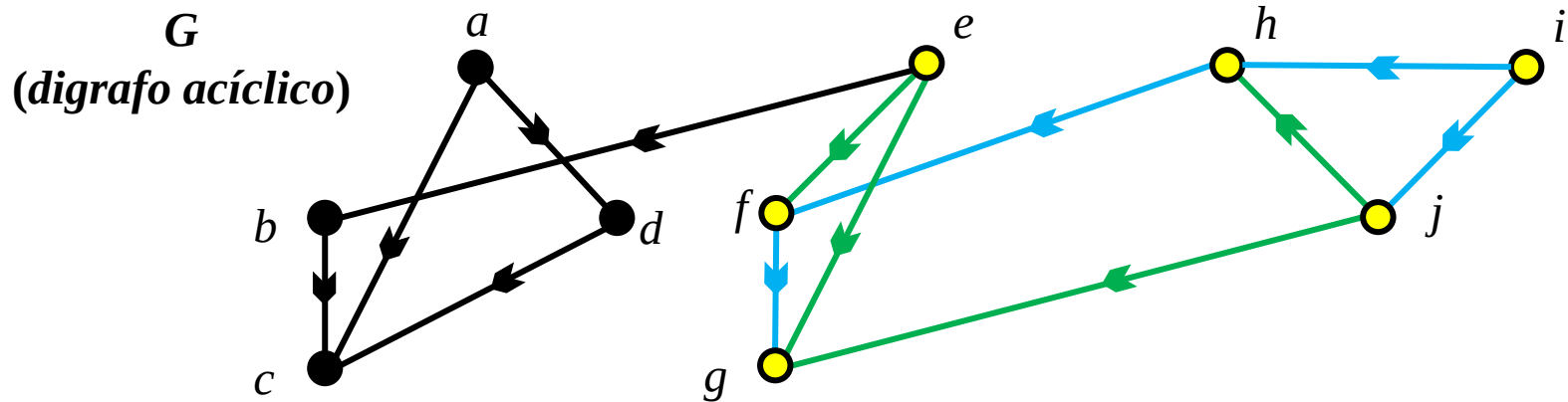
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	0	0	0	11	3	4	2	1	8
$PS(v)$	0	0	0	0	0	6	5	7	10	9

Busca em profundidade p/ digrafos

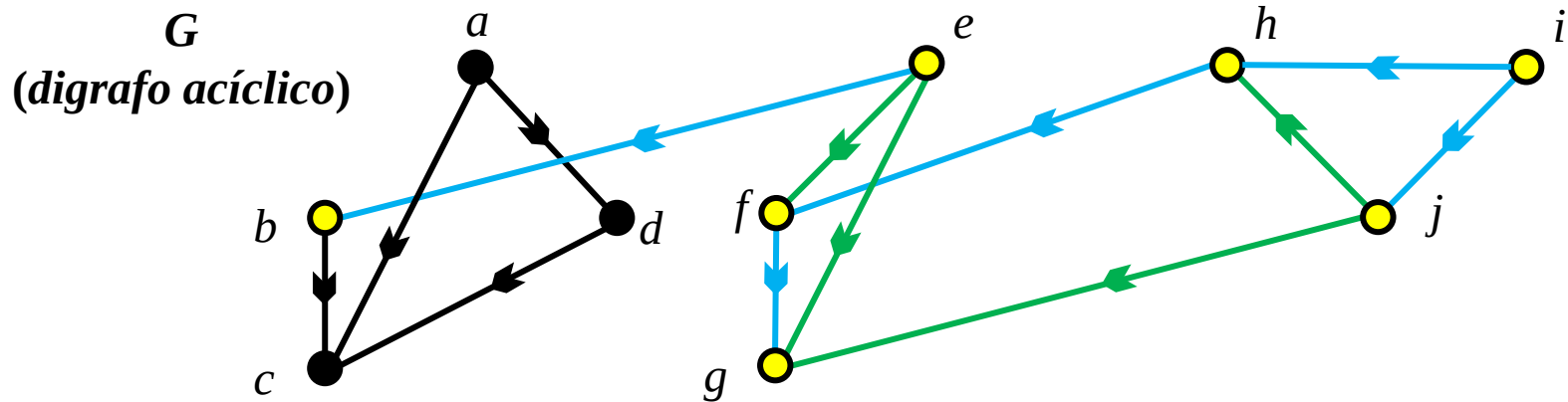
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	0	0	0	11	3	4	2	1	8
$PS(v)$	0	0	0	0	0	6	5	7	10	9

Busca em profundidade p/ digrafos

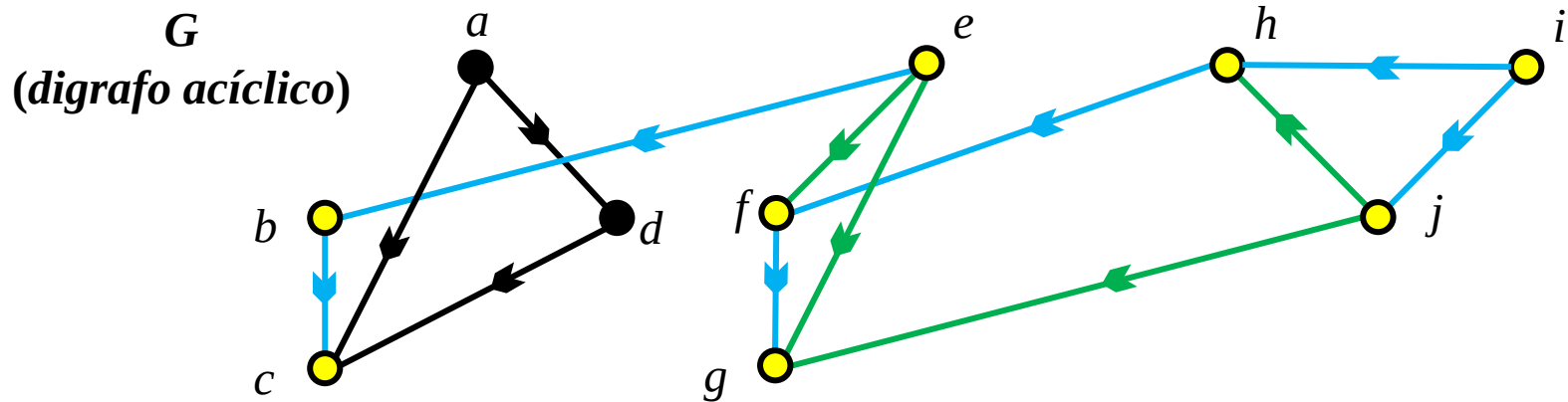
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	12	0	0	11	3	4	2	1	8
$PS(v)$	0	0	0	0	0	6	5	7	10	9

Busca em profundidade p/ digrafos

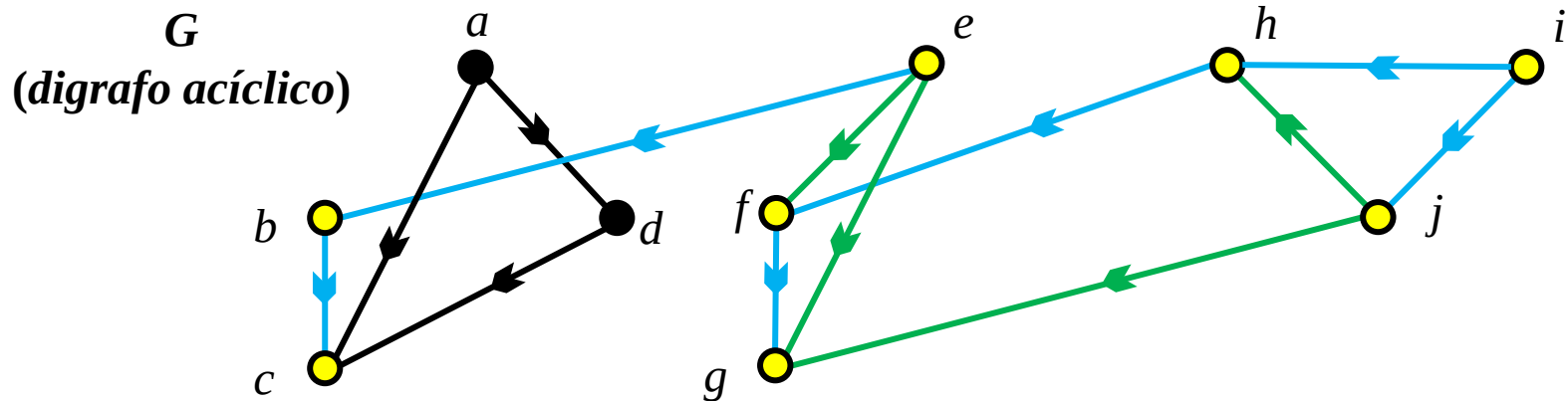
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	12	13	0	11	3	4	2	1	8
$PS(v)$	0	0	0	0	0	6	5	7	10	9

Busca em profundidade p/ digrafos

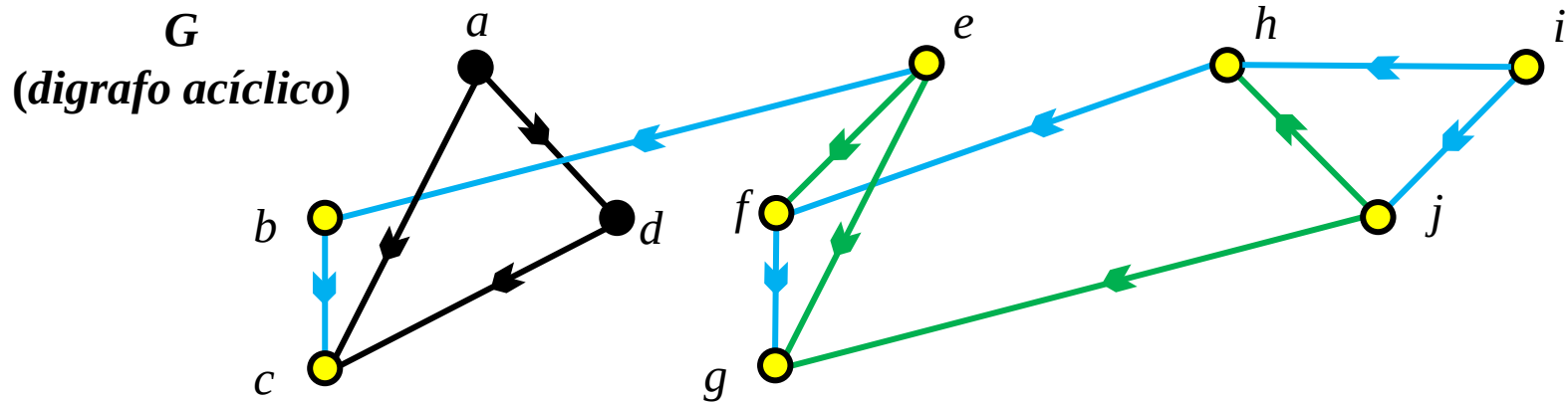
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	12	13	0	11	3	4	2	1	8
$PS(v)$	0	0	14	0	0	6	5	7	10	9

Busca em profundidade p/ digrafos

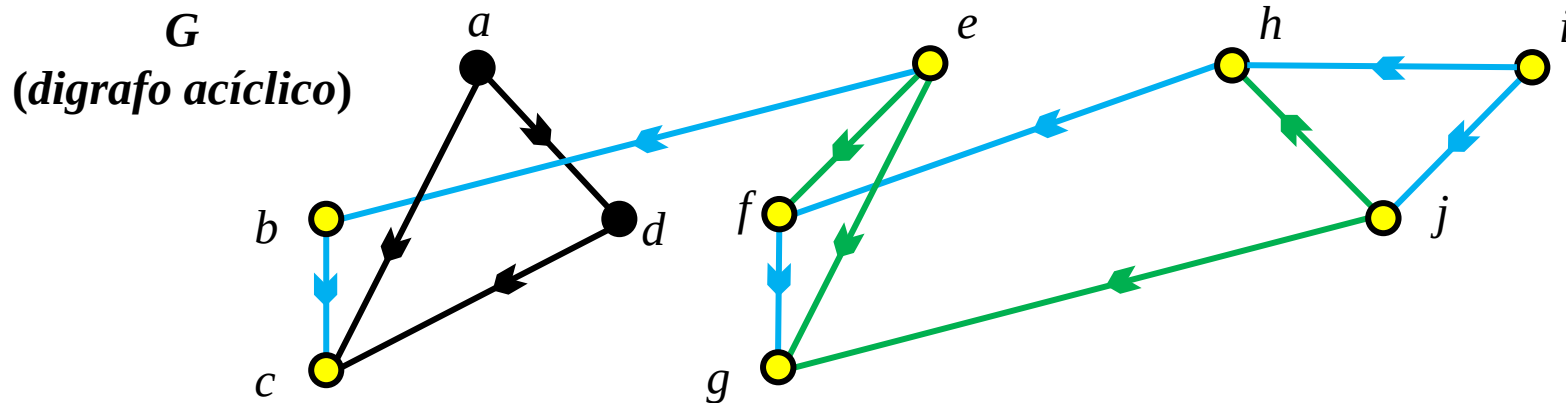
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	12	13	0	11	3	4	2	1	8
$PS(v)$	0	15	14	0	0	6	5	7	10	9

Busca em profundidade p/ digrafos

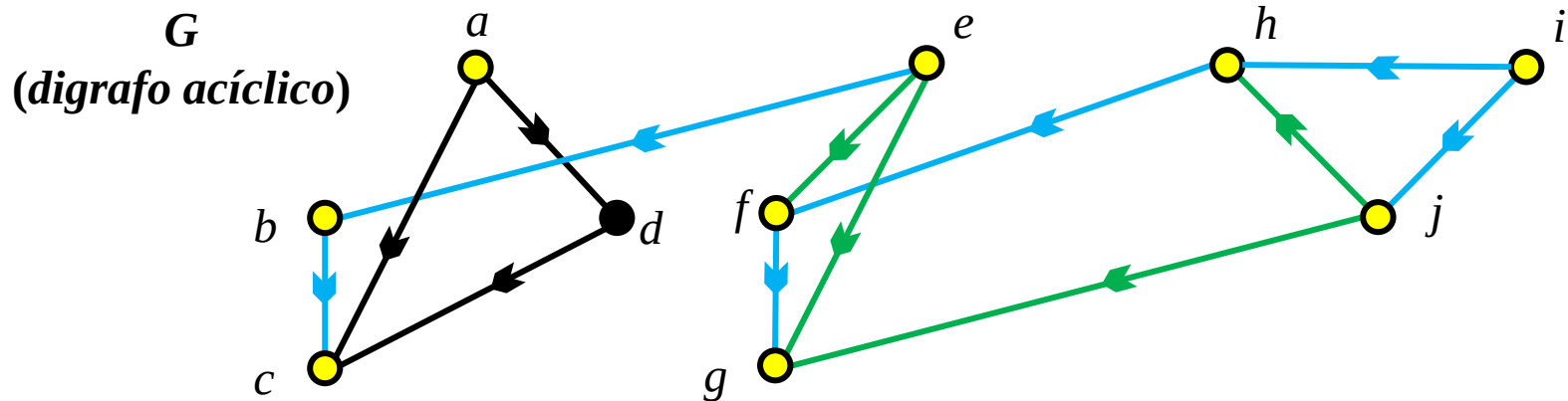
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	0	12	13	0	11	3	4	2	1	8
$PS(v)$	0	15	14	0	16	6	5	7	10	9

Busca em profundidade p/ digrafos

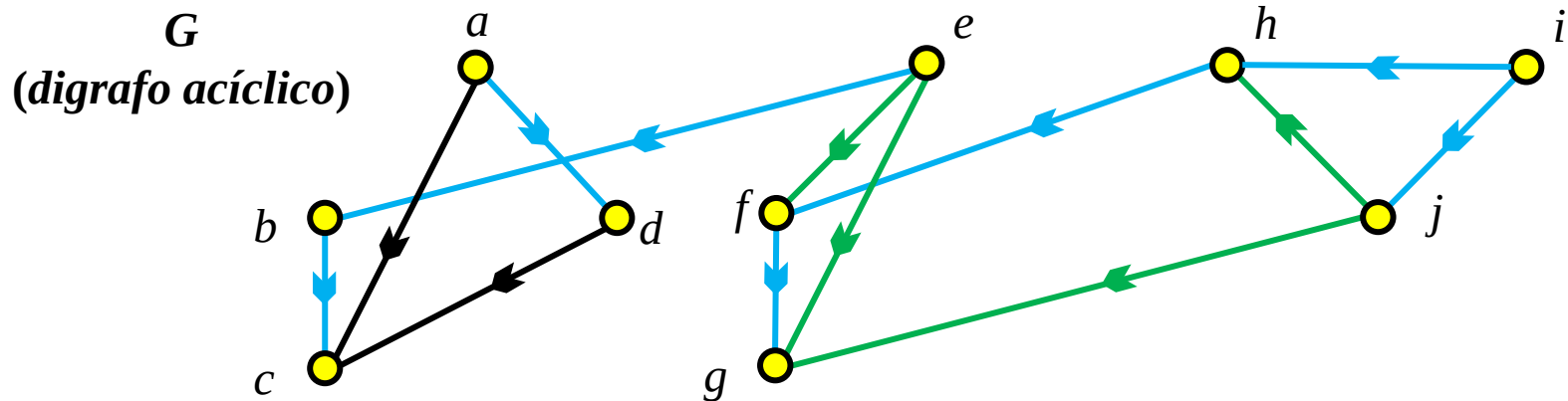
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	17	12	13	0	11	3	4	2	1	8
$PS(v)$	0	15	14	0	16	6	5	7	10	9

Busca em profundidade p/ digrafos

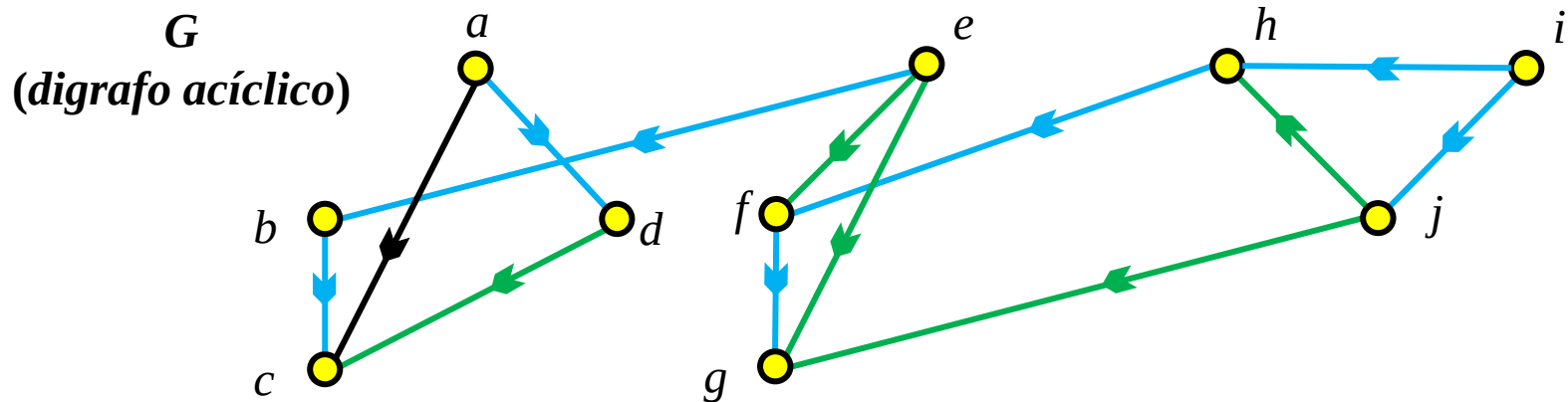
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	17	12	13	18	11	3	4	2	1	8
$PS(v)$	0	15	14	0	16	6	5	7	10	9

Busca em profundidade p/ digrafos

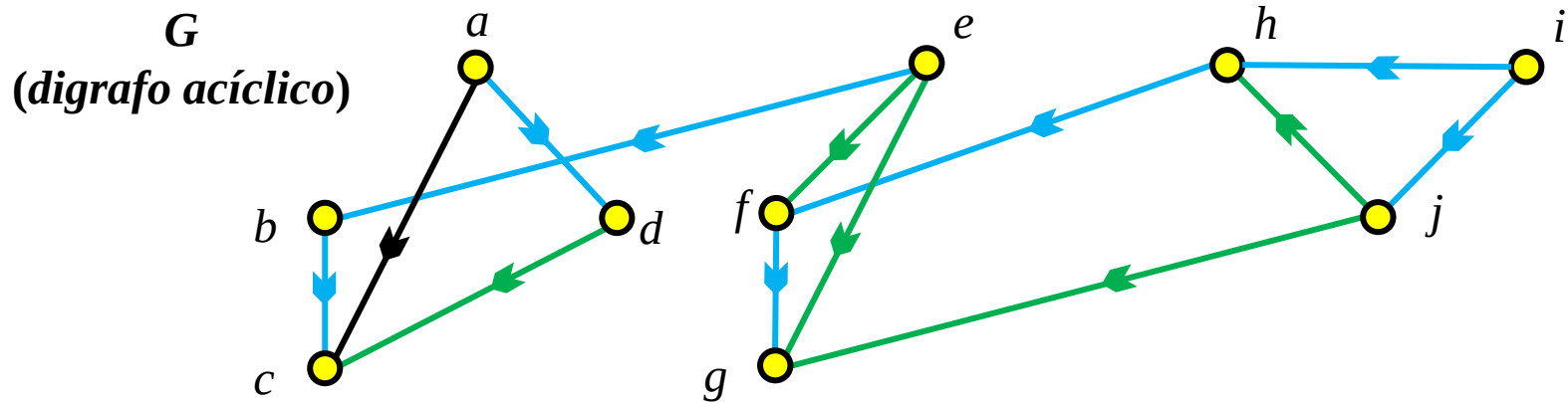
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	17	12	13	18	11	3	4	2	1	8
$PS(v)$	0	15	14	0	16	6	5	7	10	9

Busca em profundidade p/ digrafos

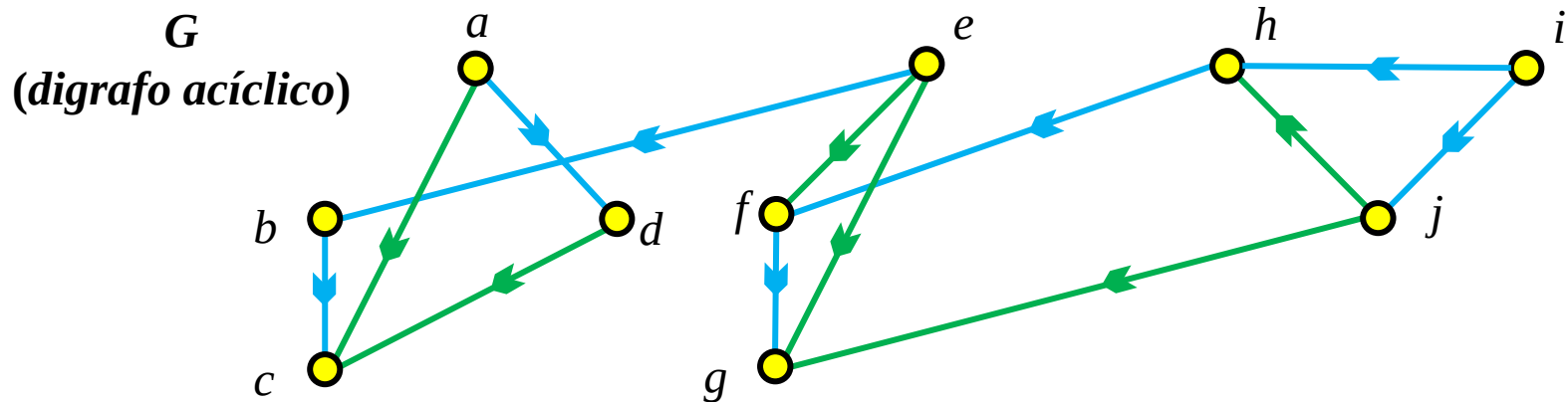
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	17	12	13	18	11	3	4	2	1	8
$PS(v)$	0	15	14	19	16	6	5	7	10	9

Busca em profundidade p/ digrafos

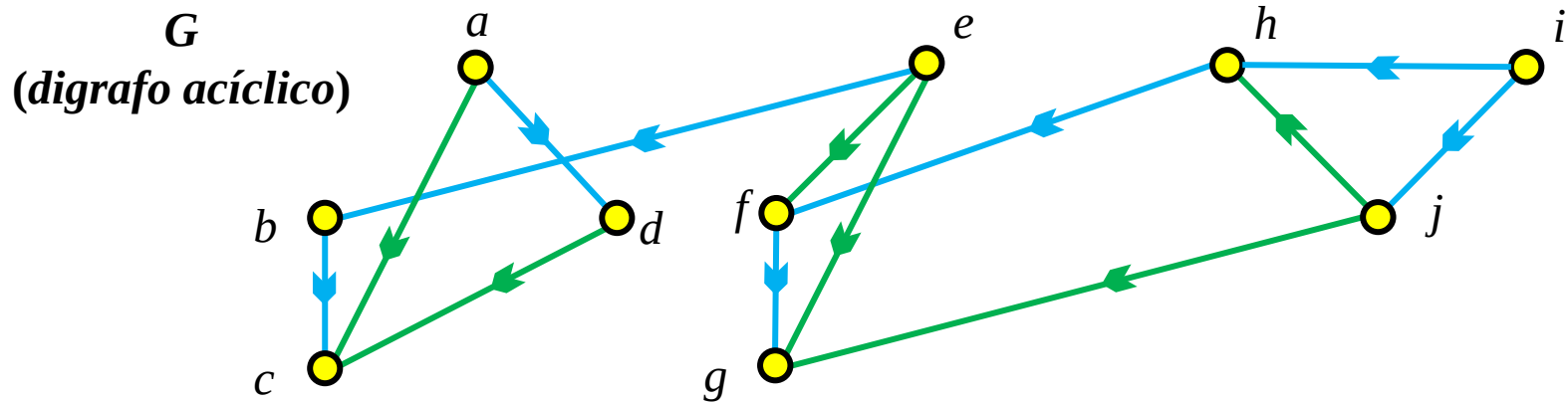
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	17	12	13	18	11	3	4	2	1	8
$PS(v)$	0	15	14	19	16	6	5	7	10	9

Busca em profundidade p/ digrafos

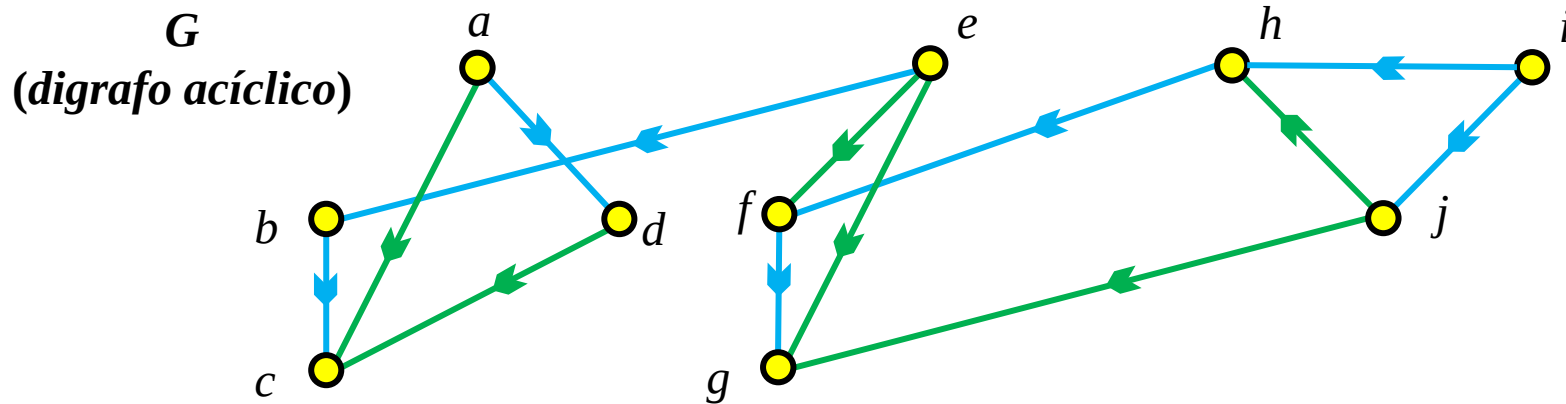
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PE(v)$	17	12	13	18	11	3	4	2	1	8
$PS(v)$	20	15	14	19	16	6	5	7	10	9

Busca em profundidade p/ digrafos

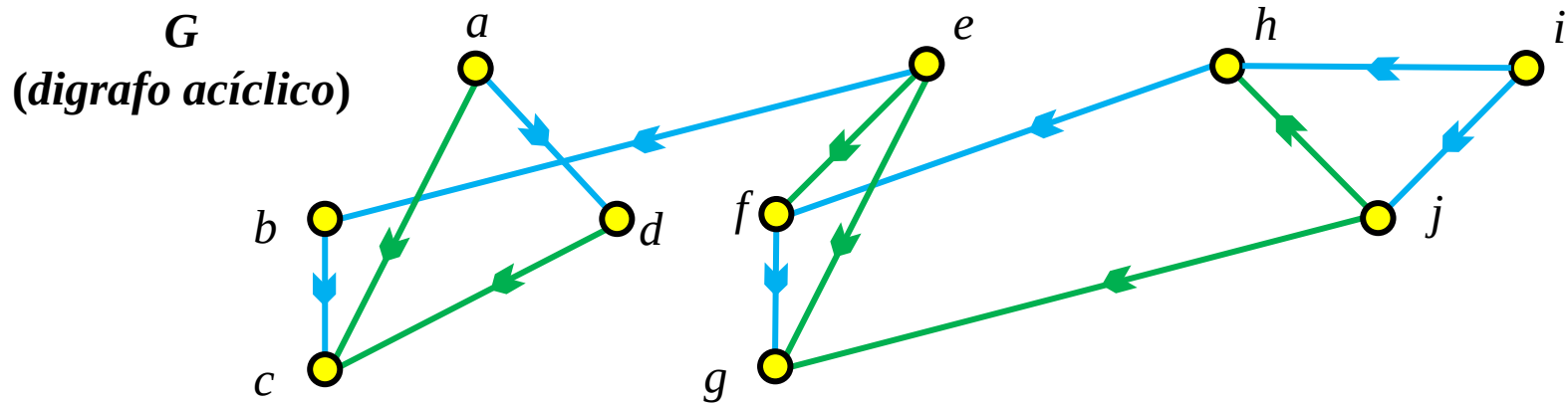
Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



v	a	b	c	d	e	f	g	h	i	j
$PS(v)$	20	15	14	19	16	6	5	7	10	9

Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .

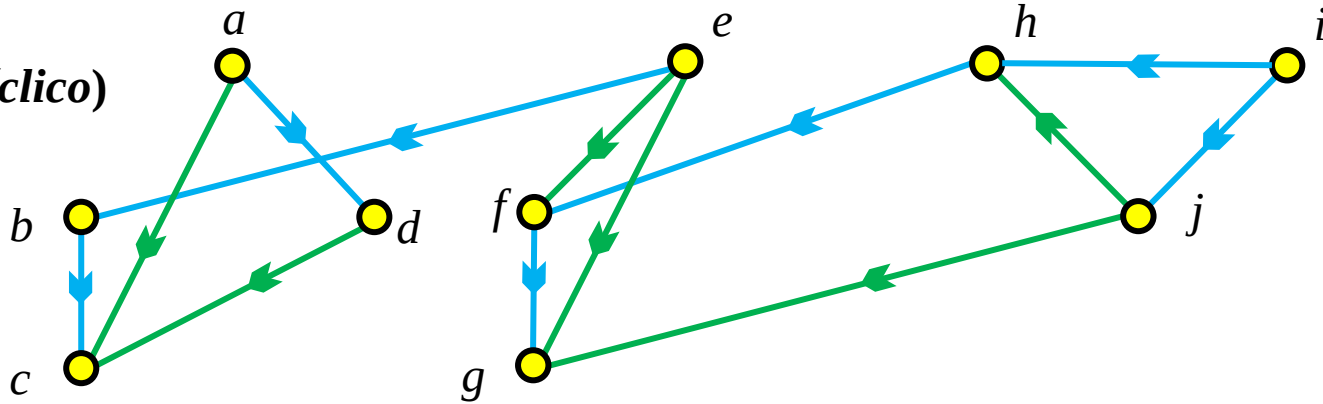


v	a	d	e	b	c	i	j	h	f	g
$PS(v)$	20	19	16	15	14	10	9	7	6	5

Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .

G
(digrafo acíclico)

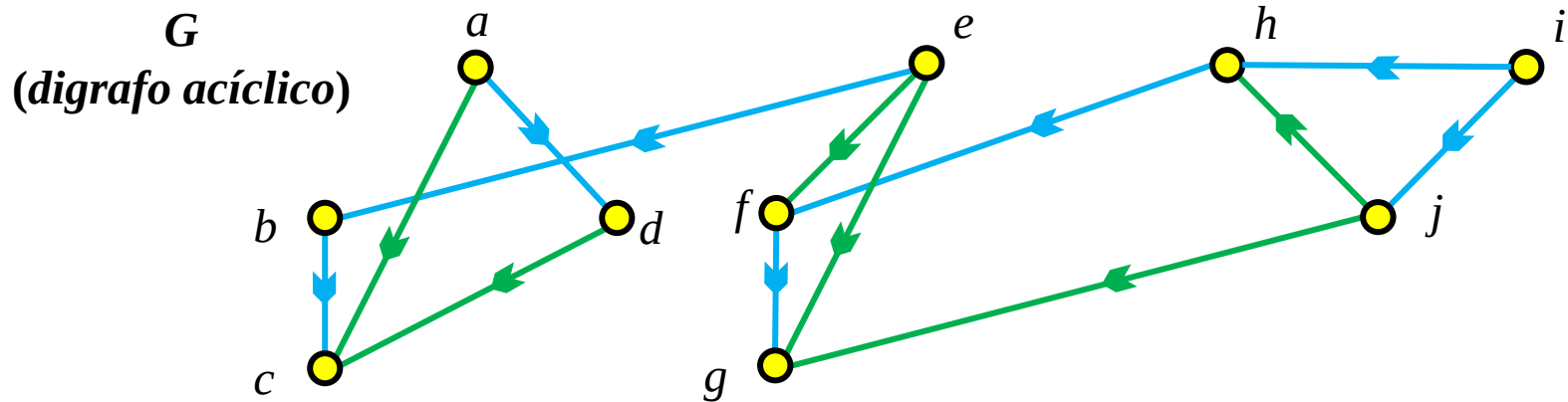


Ordenação topológica

a	d	e	b	c	i	j	h	f	g
v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9	v_{10}

Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



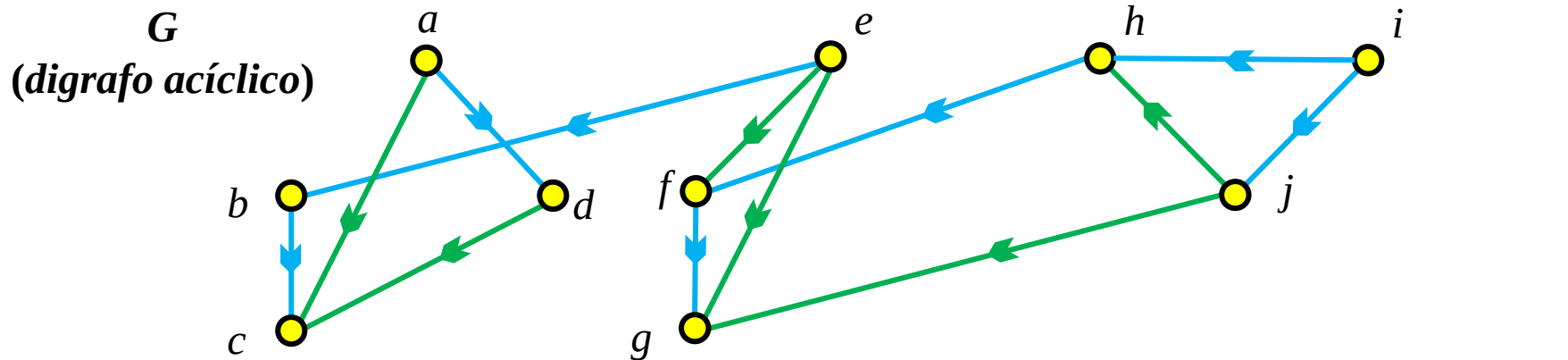
Ordenação topológica

<i>a</i>	<i>d</i>	<i>e</i>	<i>b</i>	<i>c</i>	<i>i</i>	<i>j</i>	<i>h</i>	<i>f</i>	<i>g</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------	----------

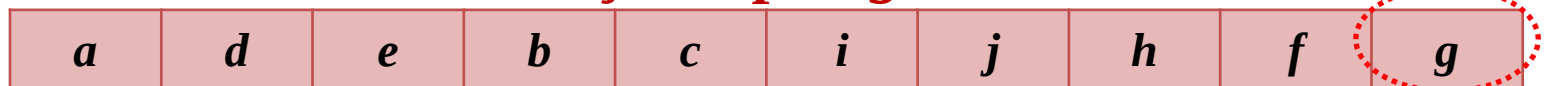
Por que este algoritmo funciona corretamente?

Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



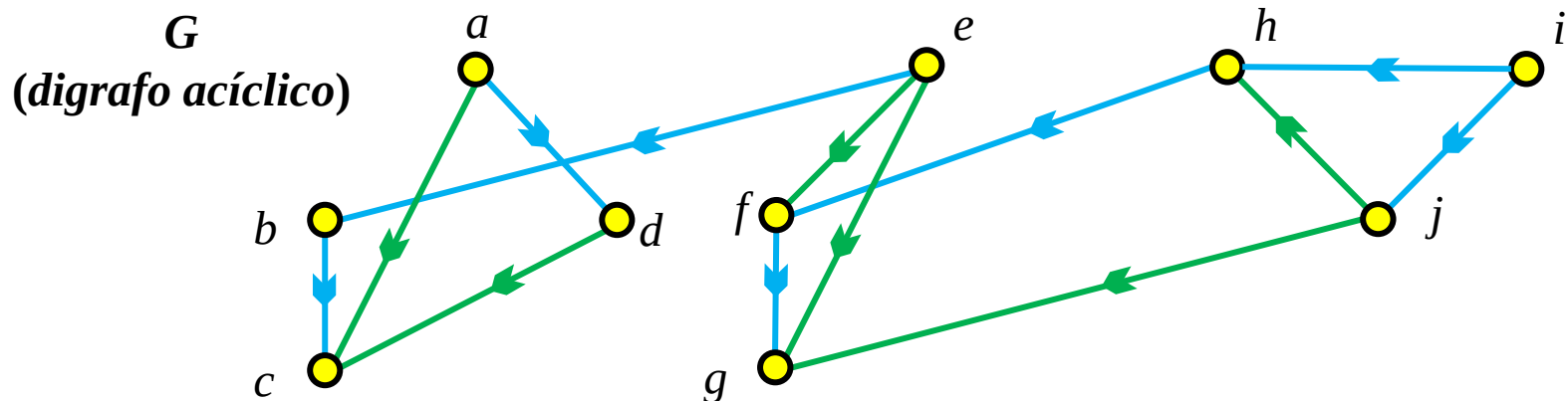
Ordenação topológica



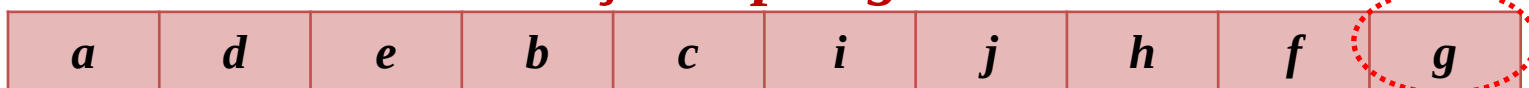
Note que o vértice com a menor PS é um **sumidouro**
(vértice com grau de saída igual a zero)

Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



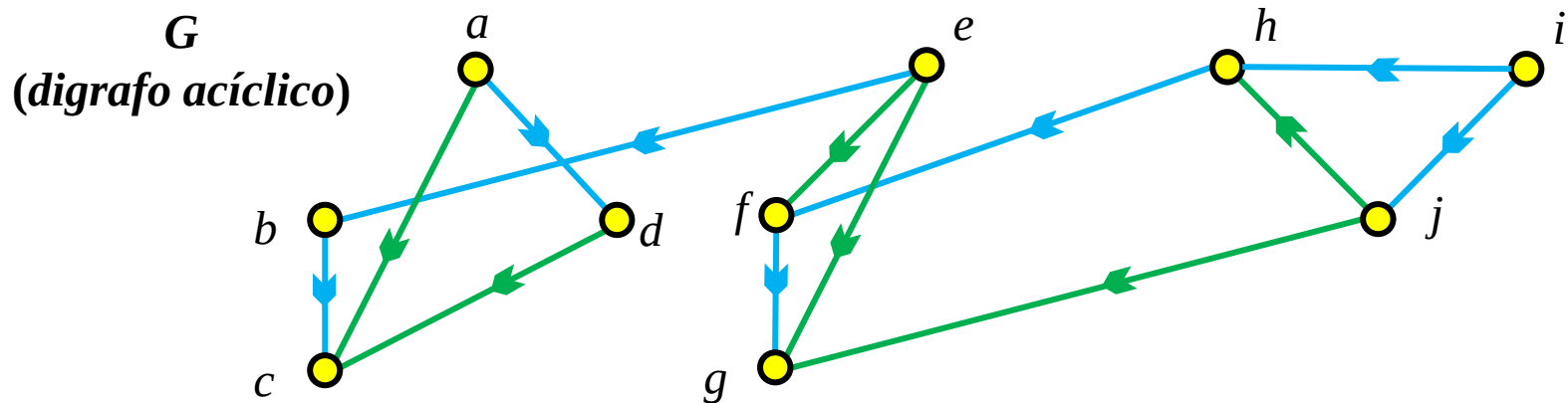
Ordenação topológica



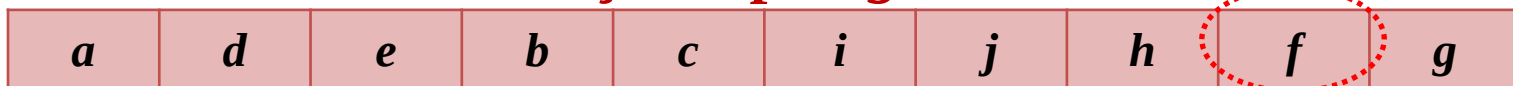
Portanto, a posição deste vértice está correta na ordenação topológica, pois dele não partem arestas

Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



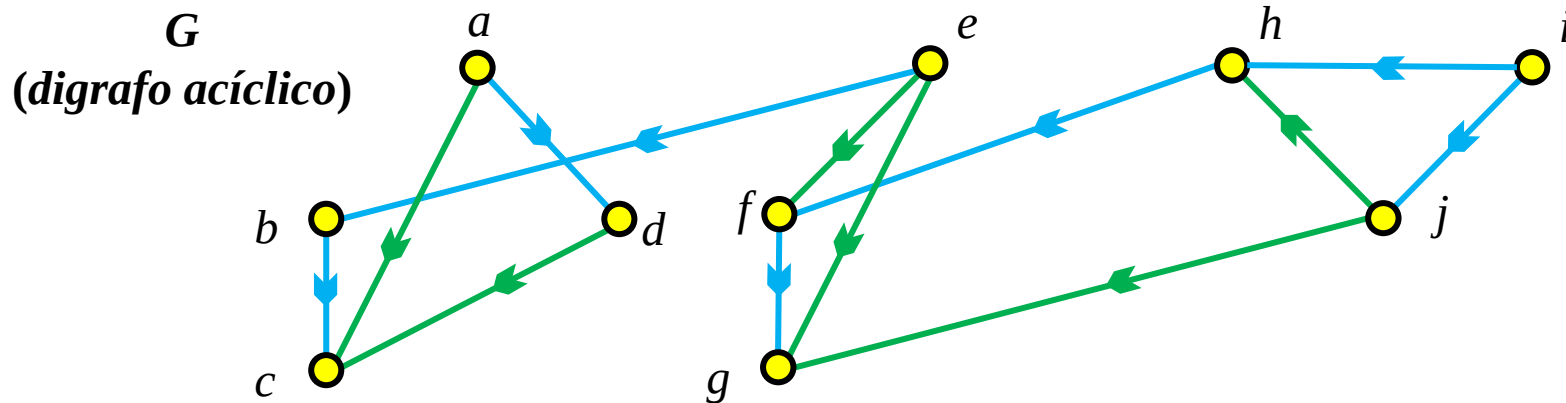
Ordenação topológica



A posição de f também está correta, pois f é um sumidouro no digrafo $G - g$

Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma ordenação topológica de G .



Ordenação topológica

a	d	e	b	c	i	j	h	f	g
v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9	v_{10}

Aplicando sucessivamente este raciocínio, cada vértice v_i é um sumidouro no digrafo $G - \{v_{i+1}, v_{i+2}, \dots, v_n\}$

Busca em profundidade p/ digrafos

Aplicação 2: Dado um digrafo acíclico G , obter uma *ordenação topológica* de G .

Complexidade do algoritmo para ordenação topológica de um digrafo G .

- Rodar uma busca em profundidade sobre G tem complexidade **$O(n + m)$** .
- Cada vez que um vértice sai da busca (recebe uma PS diferente de zero), é colocado imediatamente à esquerda da ordenação topológica sendo construída. Isto dispensa o passo de ordenação das PS 's.
- Portanto, a complexidade final é **$O(n + m)$** .

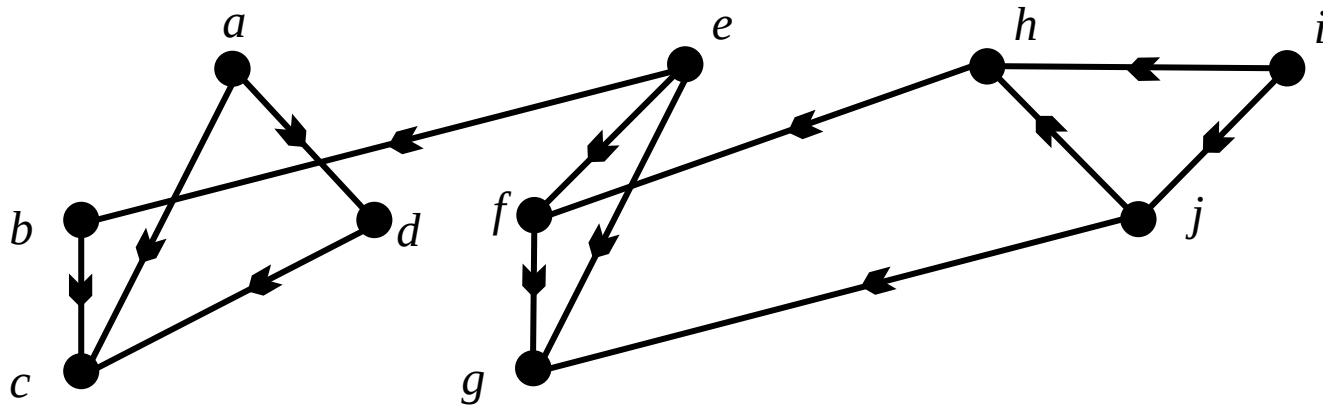
Busca em profundidade p/ digrafos

Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .

- **Idéia: todo digrafo acíclico possui um sumidouro!**
(para provar este fato, basta ver que qualquer caminho orientado de comprimento máximo em um digrafo acíclico termina necessariamente em um sumidouro)
- Realizar a seguinte iteração, até que não haja mais vértices: **localizar um sumidouro, retirá-lo do digrafo e colocá-lo à esquerda na ordenação sendo construída.**
- É possível implementar a idéia acima em tempo linear ? (isto é, $O(n + m)$?)

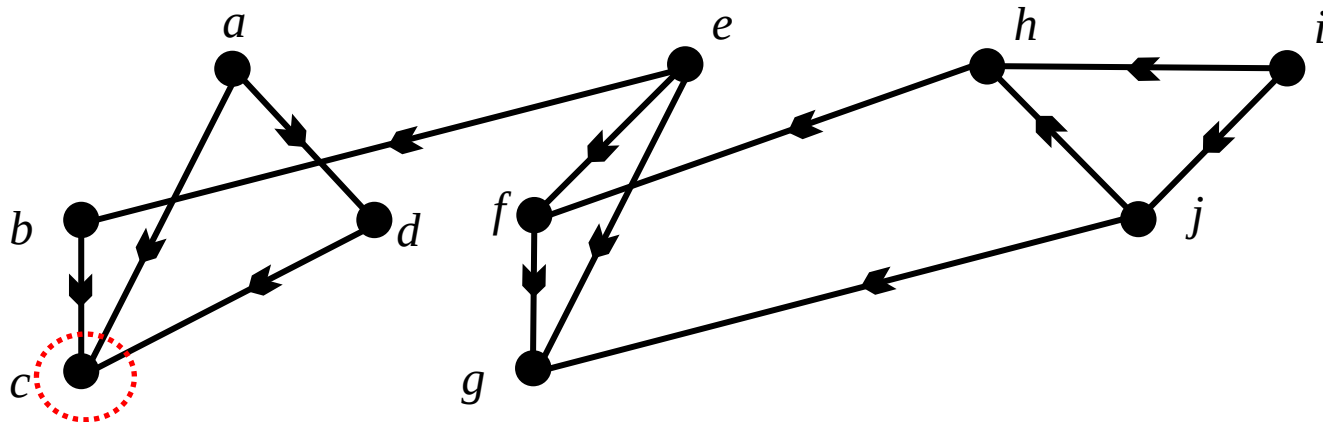
Busca em profundidade p/ digrafos

Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



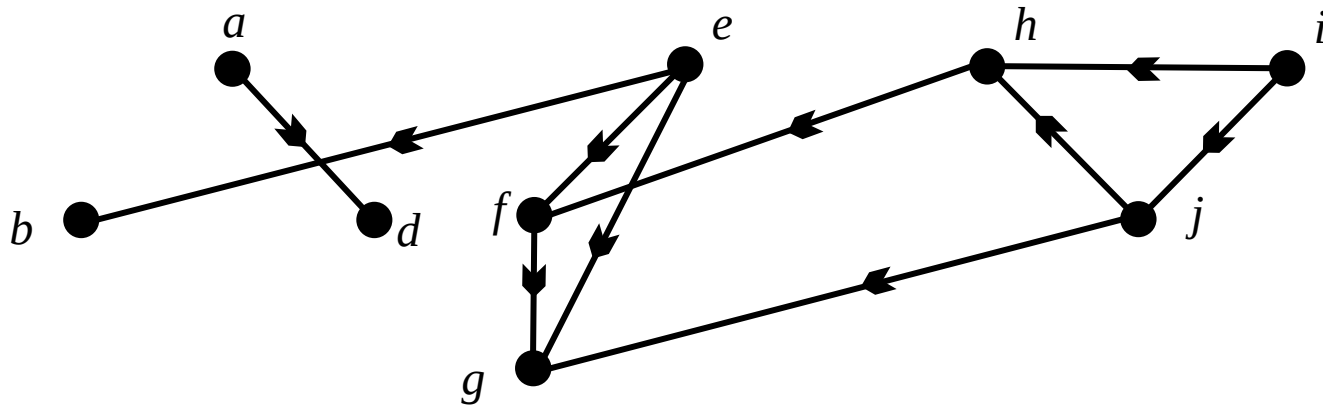
Busca em profundidade p/ digrafos

Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



Busca em profundidade p/ digrafos

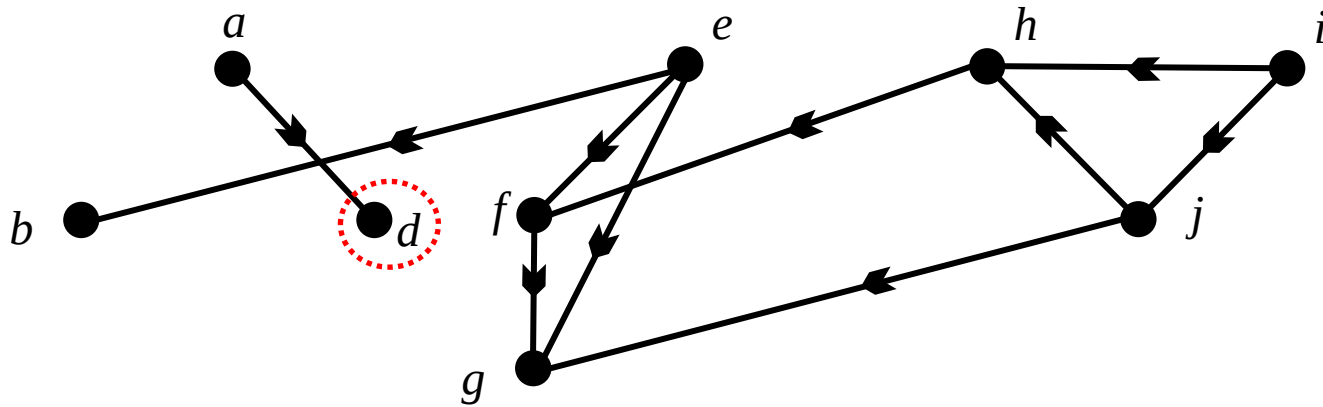
Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



c

Busca em profundidade p/ digrafos

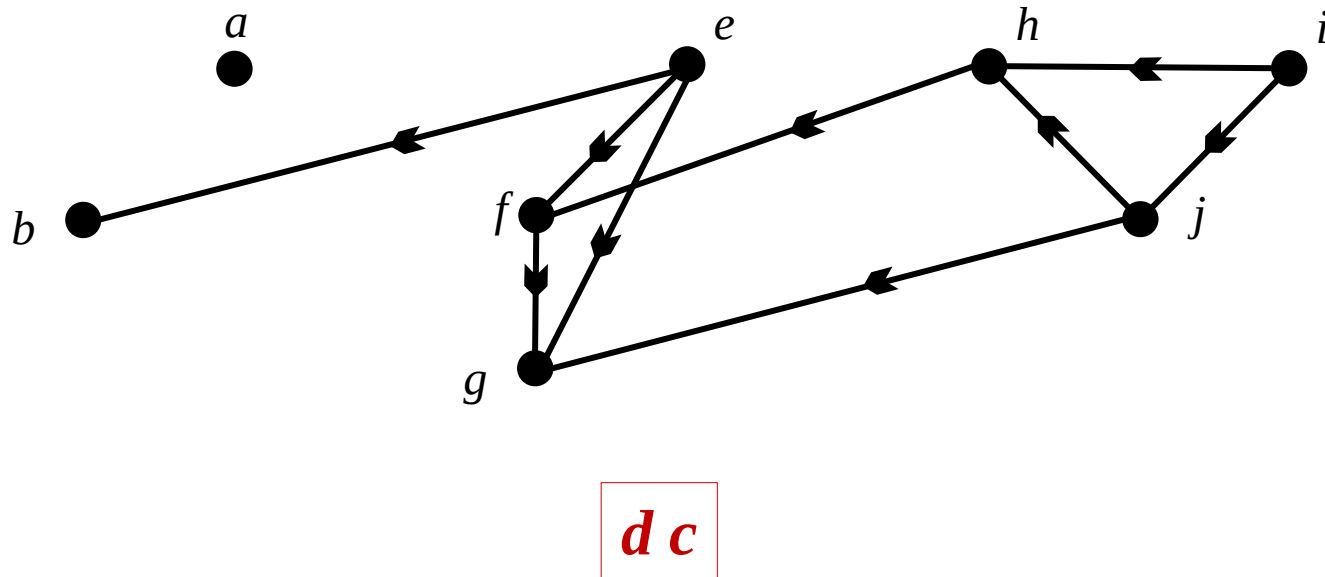
Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



c

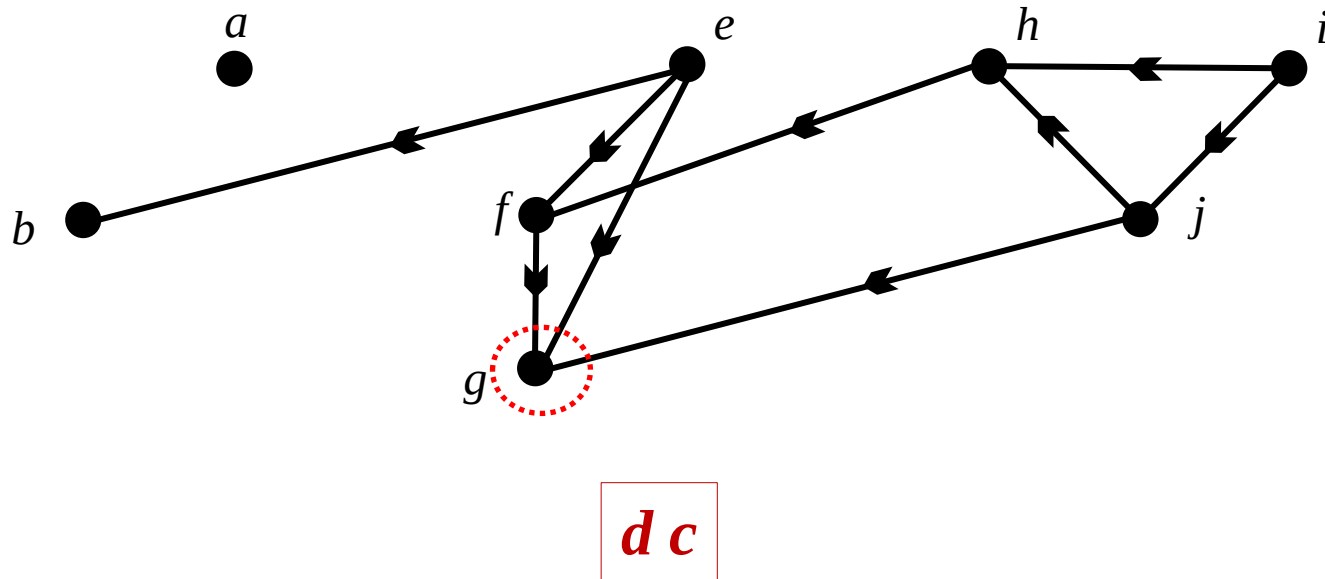
Busca em profundidade p/ digrafos

Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



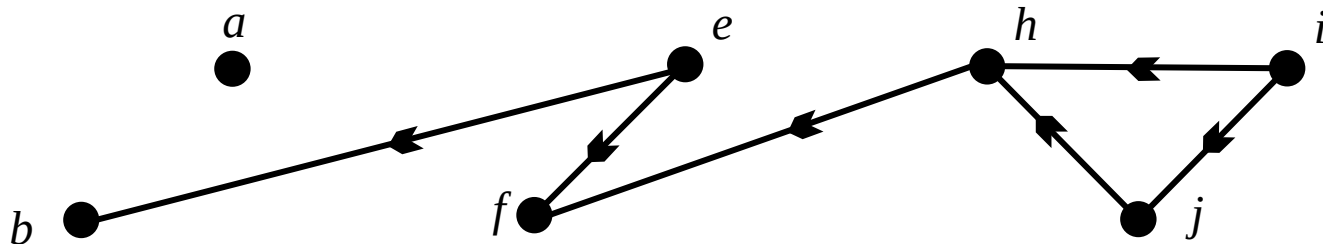
Busca em profundidade p/ digrafos

Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



Busca em profundidade p/ digrafos

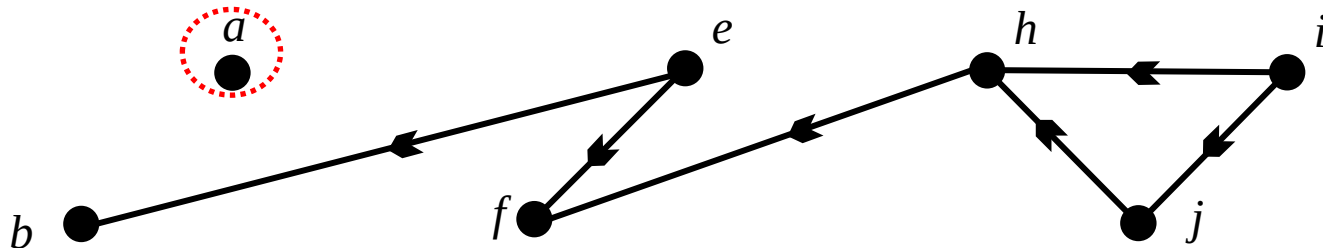
Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



g d c

Busca em profundidade p/ digrafos

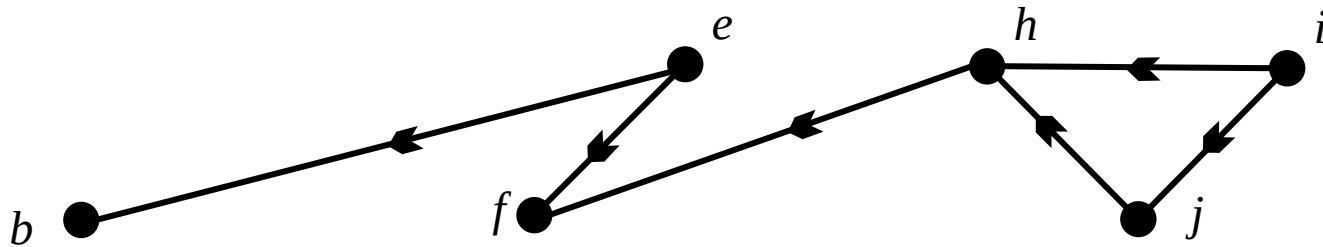
Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



g d c

Busca em profundidade p/ digrafos

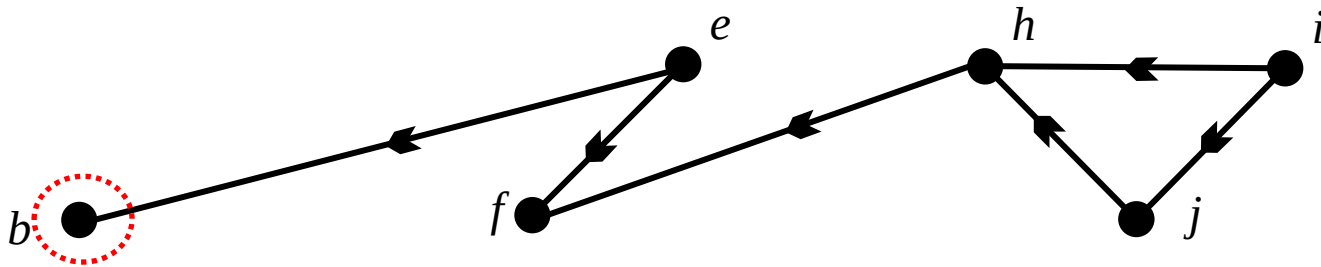
Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



a g d c

Busca em profundidade p/ digrafos

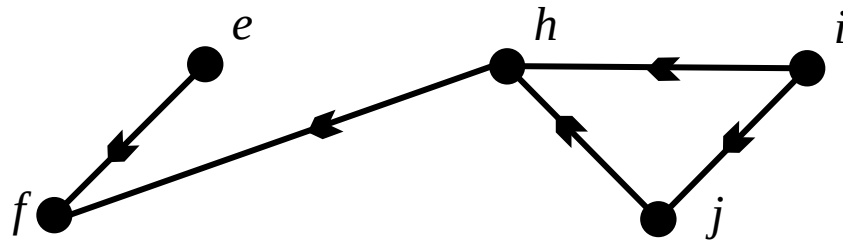
Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



a g d c

Busca em profundidade p/ digrafos

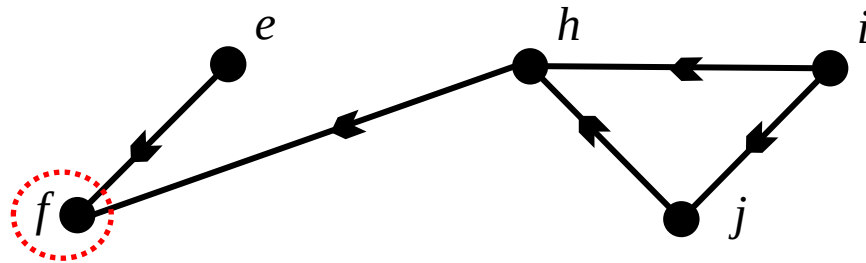
Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



b a g d c

Busca em profundidade p/ digrafos

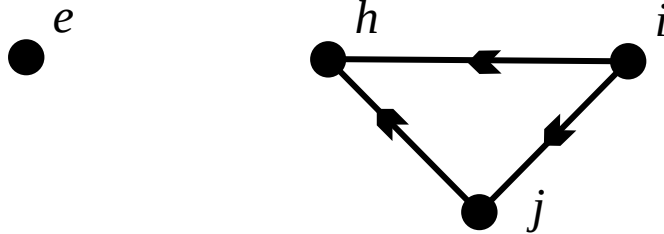
Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



b a g d c

Busca em profundidade p/ digrafos

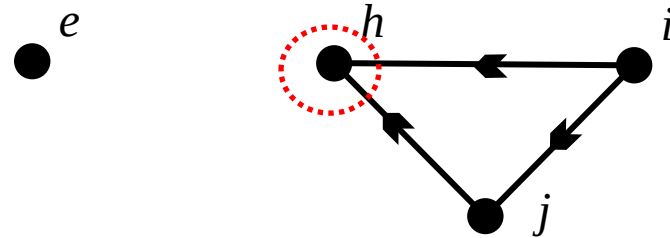
Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



f b a g d c

Busca em profundidade p/ digrafos

Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



f b a g d c

Busca em profundidade p/ digrafos

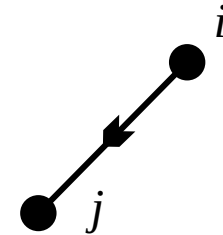
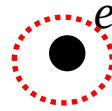
Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



h f b a g d c

Busca em profundidade p/ digrafos

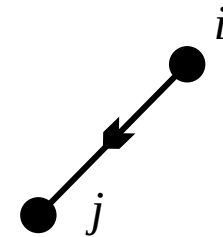
Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



h f b a g d c

Busca em profundidade p/ digrafos

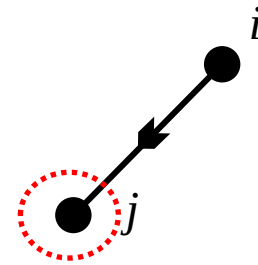
Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



e h f b a g d c

Busca em profundidade p/ digrafos

Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



e h f b a g d c

Busca em profundidade p/ digrafos

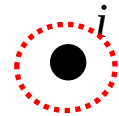
Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .

$\bullet^i$

j e h f b a g d c

Busca em profundidade p/ digrafos

Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



j e h f b a g d c

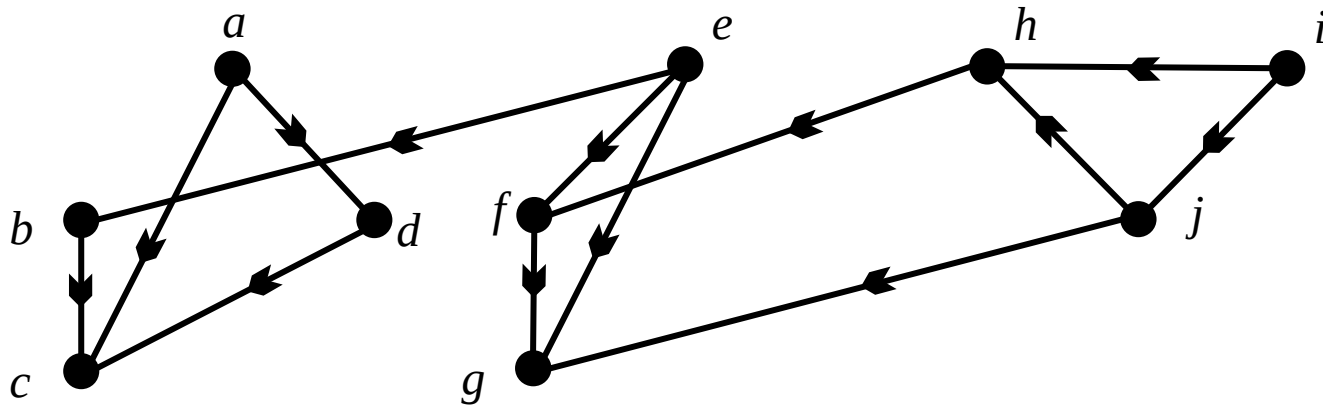
Busca em profundidade p/ digrafos

Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .

i j e h f b a g d c

Busca em profundidade p/ digrafos

Exercício: Desenvolver um algoritmo alternativo para obter uma *ordenação topológica* de um digrafo acíclico G .



i j e h f b a g d c

Ordenação topológica!